

05-git-deploy

05. Git & GitHub + Deploy

Level: □ Beginner

Jam: 4 (2 minggu × 2 sesi)

Prasyarat: JavaScript Fundamentals

Output: Semua project di GitHub + landing page live

Tujuan Pembelajaran

Setelah modul ini, kamu bisa:

- Pake Git buat version control project sendiri
- Commit, branch, merge kayak dev profesional
- Kolaborasi lewat GitHub (push, pull, pull request)
- Deploy frontend ke Vercel
- Deploy backend ke Railway
- Atur environment variable & custom domain

Materi

Sesi	Topik	File
1	Git Basics — init, add, commit, log, reset, .gitignore	01-git-basics.md
2	GitHub Collaboration — remote, push, pull, branch, merge, PR, conflict	02-github-collab
3	Deploy — Vercel, Railway, env vars, custom domain	03-deploy.md

Output Akhir Modul

GitHub Repo + Landing Page Live — repo berisi project landing page, terdeploy di Vercel, bisa diakses publik

AI Prompt Exercises

Sepanjang modul, latihan pake AI:

- "Explain this git command and what it does in detail"
- "I got a merge conflict in this file — help me resolve it"
- "Generate a .gitignore file for a Node.js project"
- "Review my commit history and tell me if the messages are good"

1.1 Git Basics

Inisialisasi Repository

```
# Bikin folder project dan masukin git
mkdir project-saya
cd project-saya
git init
# Output: Initialized empty Git repository in .../.git/
```

Setelah git init, folder .git muncul — isinya semua riwayat versi. Jangan dihapus.

Cek Status

```
# Cek file mana aja yang berubah
git status
# Output: untracked files, modified files, staged files
```

Staging & Commit

```
# Bikin file
echo "<h1>Halo Dunia</h1>" > index.html

# Stage – tandain file yang mau di-commit
git add index.html

# Atau stage semua file
git add .

# Commit – simpan snapshot
git commit -m "feat: tambah index.html"
```

Melihat Riwayat (Log)

```
# Lihat semua commit
git log

# Format pendek (1 line per commit)
```

```
git log --oneline  
  
# Lihat grafis branch  
git log --oneline --graph --all
```

Melihat Perubahan (Diff)

```
# Apa aja yang berubah sebelum di-stage  
git diff  
  
# Apa aja yang udah di-stage (siap di-commit)  
git diff --staged  
  
# Bandingin dua commit tertentu  
git diff abc123 def456
```

Undo — Reset

```
# Unstage file (balikin dari staging area)  
git reset HEAD index.html  
  
# Balikin ke commit sebelumnya, file tetap ada (soft reset)  
git reset --soft HEAD~1  
  
# Balikin ke commit sebelumnya, file hilang (hard reset – HATI-HATI!)  
git reset --hard HEAD~1  
  
# Balikin file ke versi commit tertentu  
git restore index.html
```

.gitignore — File yang Diabaikan

Bikin file .gitignore di root project:

```
# Folder dependencies  
node_modules/  
vendor/  
  
# File environment  
.env  
.env.local  
  
# File build  
dist/  
build/
```

```
.next/

# File OS
.DS_Store
Thumbs.db

# Log
*.log
npm-debug.log*

# Pastiin .gitignore ke-track juga
git add .gitignore
git commit -m "chore: tambah .gitignore"
```

Latihan

1. Bikin folder latihan-git, git init, bikin index.html, git add & git commit
2. Ubah index.html, cek git diff, git add lagi, git commit
3. Bikin .gitignore, isi node_modules/ dan .env, commit
4. Bikin 3 commit, cek git log --oneline, undo commit kedua pake git reset --soft HEAD~1

1.2 GitHub Collaboration

Setup Remote

Bikin repo dulu di github.com (jangan centang README).

```
# Hubungkan repo lokal ke GitHub
git remote add origin https://github.com/username/nama-repo.git

# Kirim branch utama ke GitHub
git push -u origin main
```

-u artinya set upstream — setelah ini cukup pake git push aja.

Clone & Pull

```
# Download repo dari GitHub ke lokal (baru pertama)
git clone https://github.com/username/nama-repo.git

# Ambil update terbaru (udah pernah clone)
git pull origin main
```

Branch — Bikin Cabang

```
# Lihat branch saat ini
git branch

# Bikin branch baru
git branch fitur-login

# Pindah ke branch itu
git checkout fitur-login

# Bikin + pindah sekaligus
git checkout -b fitur-login
```

Merge — Gabung Branch

```
# Selesai ngerjain fitur, gabung ke main
git checkout main
git merge fitur-login

# Hapus branch yang udah di-merge (optional)
git branch -d fitur-login
```

Pull Request (PR) — Kolaborasi Tim

Langkah-langkah PR di GitHub:

1. Push branch ke GitHub:

```
git checkout -b fitur-navbar
git add .
git commit -m "feat: tambah navbar responsive"
git push -u origin fitur-navbar
```
2. Buka GitHub → muncul tombol **Compare & pull request**
3. Klik, tulis deskripsi, klik **Create pull request**
4. Minta temen review
5. Klik **Merge pull request** setelah disetujui
6. Pull main lagi di lokal:

```
git checkout main
git pull origin main
```

Conflict Resolution — Konflik Merge

Conflict terjadi saat 2 orang ubah baris yang sama di file yang sama.

```
# Contoh situasi conflict
git checkout -b ubah-warna
```

```
# ubah teks di index.html
git add .
git commit -m "style: ganti warna header"
```

```
git checkout main
# ubah baris YANG SAMA di index.html
git add .
git commit -m "fix: perbaiki typo header"
```

```
git merge ubah-warna
# Output: CONFLICT (content) in index.html
```

Di file yang conflict, Git kasih penanda:

```
<<<<<<< HEAD
<h1 style="color: red">Selamat Datang</h1>
=====
<h1 style="color: blue">Selamat Datang</h1>
>>>>>>> ubah-warna
```

Langkah resolve:

1. Buka file di editor
2. Hapus <<<<<<< HEAD, =====, >>>>>>> ubah-warna
3. Pilih versi yang bener (atau gabungin)
4. Simpan, lalu:

```
git add index.html
git commit -m "merge: resolve conflict warna header"
```

Git Workflow Standar Tim

```
# 1. Ambil update terbaru
git checkout main
git pull origin main
```

```
# 2. Bikin branch fitur
git checkout -b fitur-saya
```

```
# 3. Kerja, commit, push
git add .
git commit -m "feat: ..."
git push -u origin fitur-saya
```

```
# 4. Buka PR di GitHub → review → merge
# 5. Balik ke main, update, hapus branch lokal
git checkout main
git pull origin main
```

```
git branch -d fitur-saya
```

Latihan

1. Bikin repo baru di GitHub, git remote add origin, push project dari sesi 1
2. Bikin branch fitur-footer, ubah footer di index.html, commit, push, bikin PR di GitHub, merge
3. Simulasi conflict: 2 branch ubah baris yang sama di README, resolve conflict pake editor
4. Clone repo punya temen, bikin branch, commit, push, bikin PR (minta temen review & merge)

1.3 Deploy — Frontend & Backend

Deploy Frontend ke Vercel

Persiapan

Bikin project frontend (statis atau React/Next.js):

```
# Bikin folder landing page
```

```
mkdir landing-page
```

```
cd landing-page
```

```
# Bikin file index.html
```

```
cat > index.html << 'EOF'
```

```
<!DOCTYPE html>
```

```
<html lang="id">
```

```
<head>
```

```
  <meta charset="UTF-8" />
```

```
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
```

```
  <title>Portfolio Saya</title>
```

```
  <link rel="stylesheet" href="style.css" />
```

```
</head>
```

```
<body>
```

```
  <h1>Halo! Saya <span id="nama">Budi</span></h1>
```

```
  <p>Ini landing page pertama saya <span></span></p>
```

```
  <script src="script.js"></script>
```

```
</body>
```

```
</html>
```

```
EOF
```

```
# Bikin style.css
```

```
cat > style.css << 'EOF'
```

```

body {
  font-family: system-ui, sans-serif;
  max-width: 640px;
  margin: 80px auto;
  text-align: center;
  background: #0f172a;
  color: #f8fafc;
}
h1 { font-size: 2.5rem; }
span { color: #38bdf8; }
EOF

# Bikin script.js
cat > script.js << 'EOF'
const namaSpan = document.getElementById('nama');
const nama = prompt('Siapa nama kamu?') || 'Budi';
namaSpan.textContent = nama;
EOF

```

Push ke GitHub

```

git init
git add .
git commit -m "feat: landing page portfolio"
# Ganti dengan username & repo kamu
git remote add origin https://github.com/username/landing-page.git
git push -u origin main

```

Deploy di Vercel

1. Buka vercel.com, login pake GitHub
2. Klik **Add New** → **Project**
3. Pilih repo landing-page
4. Settings default (Vercel detect HTML otomatis)
5. Klik **Deploy**
6. Selesai! Dapet URL: <https://landing-page.vercel.app>

Deploy Backend ke Railway

Persiapan

Bikin API sederhana pake Node.js + Express:

```

mkdir api-saya
cd api-saya

```



```

npm init -y
npm install express cors

Bikin index.js:

const express = require('express');
const cors = require('cors');
const app = express();
const PORT = process.env.PORT || 3000;

app.use(cors());
app.use(express.json());

app.get('/api/halo', (req, res) => {
  res.json({ pesan: 'Halo dari Railway!' });
});

app.get('/api/users', (req, res) => {
  res.json([
    { id: 1, nama: 'Budi' },
    { id: 2, nama: 'Andi' }
  ]);
});

app.listen(PORT, () => {
  console.log(`Server jalan di port ${PORT}`);
});

Bikin start script di package.json:

"scripts": {
  "start": "node index.js"
}

Bikin .gitignore:

node_modules/
.env

```

Push & Deploy

```

git init
git add .
git commit -m "feat: api express sederhana"
git remote add origin https://github.com/username/api-saya.git
git push -u origin main

```

1. Buka railway.com, login pake GitHub
2. Klik **New Project** → **Deploy from GitHub repo**

3. Pilih repo api-saya
4. Railway auto-detect Node.js, jalanin `npm install + start`
5. Dapet URL: `https://api-saya.up.railway.app`

Test API:

```
curl https://api-saya.up.railway.app/api/halo  
# Output: {"pesan":"Halo dari Railway!"}
```

Environment Variable

Jangan pernah hardcode secret di kode! Pake env variable.

Local Development

Bikin `.env` (jangan di-commit — masukin `.gitignore`):

```
DB_HOST=localhost  
DB_USER=admin  
DB_PASS=rahasia123  
API_KEY=sk-xxx
```

Baca di code:

```
// Pakai dotenv (install: npm install dotenv)  
require('dotenv').config();  
  
const dbPass = process.env.DB_PASS;  
const apiKey = process.env.API_KEY;  
  
console.log(dbPass); // rahasia123
```

Di Vercel

Dashboard project → **Settings** → **Environment Variables** → isi key-value.

Di Railway

Dashboard project → **Variables** → **New Variable** → isi key-value.

Railway juga bisa upload `.env` file langsung.

Custom Domain

Vercel

Dashboard project → **Settings** → **Domains** → masukan domain kamu (misal portofolio-saya.com) → ikutin instruksi DNS (CNAME record).

Railway

Dashboard project → **Settings** → **Domains** → **Custom Domain** → ikutin instruksi DNS.

Rekomendasi Domain Gratis / Murah

- [Niagahoster](#) — .com ~150rb/thn
- [Cloudflare Registrar](#) — harga modal (+- 90rb/thn)
- [Freenom](#) — .tk, .ml, .ga (gratis, kadang bermasalah)

Latihan

1. Bikin landing page (HTML+CSS+JS), push ke GitHub, deploy ke Vercel
2. Bikin Express API dengan 2 endpoint, deploy ke Railway, test pake curl
3. Tambah env variable SITE_NAME di Vercel, pake di landing page, re-deploy
4. Hubungin custom domain (bisa pake subdomain gratisan kaya namamu.vercel.app)